

PROJECT X

Field Photo & Geolocation Logging Suite

Technical Documentation & Engineering Report

Prepared by: Rajiv M

Gmail Id: mrjiv4969@gmail.com

Assessment: Technical Internship Assessment — Round 2

Submission deadline: 30 September 2026

Technology stack: React 18, Vite 5, Node.js, Express 4, Google Drive API v3, Google Sheets API v4, Leaflet / OpenStreetMap, exifr

Repository: github.com/rajiv-1004/Project-X

Evaluation account: su1@vr2.in (Drive permission verification target)

Video demonstration: [Output_Proof](#)

This report documents the application, its architecture, the AI-assisted development process, Google API configuration, verification evidence, engineering challenges, and planned improvements. Every factual claim in this document has been checked directly against the current source repository and running application, rather than restated from an AI tool's own summary of its work — this distinction, and why it matters, is the subject of Section 5.

Contents

1. Executive Summary	page - 3
2. Assessment Requirements and Implementation	page - 4
3. System Architecture and Component Interaction	page - 5
4. Application Workflow and Data Handling	page - 6
5. AI Collaboration: Build Process and Independent Verification	page - 7
6. Agent Skills and Model Selection	page - 10
7. Google Cloud, OAuth, Drive and Sheets Configuration	page - 11
8. Security, Quality and Engineering Standards Compliance	page - 12
9. Application Evidence and Visual Walkthrough	page - 13
10. Challenges and Resolutions	page - 17
11. Future Enhancements	page - 17
12. Conclusion and Submission Checklist	page - 18

1. Executive Summary

Project X is a full-stack React application built for field surveying, environmental inspection, and geospatial photo logging. A user authenticates with Google, uploads or captures photographs, has GPS coordinates extracted from EXIF metadata in the browser, and has that data stored in a user-specific Google Drive folder and logged to a Google Sheet. The user can browse uploaded assets in a gallery, inspect geotagged photographs on an interactive map, and share individual files with another Google account through Drive permissions rather than a public link.

The application separates a React single-page frontend from a Node.js / Express API gateway. All Google API operations and sensitive credentials remain on the backend; EXIF GPS parsing happens client-side before upload, while the backend handles OAuth code exchange, Drive operations, Sheets logging, and authenticated image streaming.

Verification basis for this report. Every architectural and code-level claim in this document was checked directly against the live GitHub repository at the time of writing — by inspecting the actual source files, not by restating a development tool's own description of its work. Two inaccuracies from an earlier draft of this report were caught and corrected this way before finalization (Section 7.3 notes one; both are detailed for transparency).

1.1 Project Objectives

The project demonstrates the ability to decompose a practical assignment into manageable requirements, direct AI tools through iterative and verifiable prompts, integrate third-party APIs correctly, validate behavior against the actual running system rather than a tool's self-report, and document engineering decisions transparently — including where something did not work on the first attempt.

1.2 Application at a Glance

Layer	Responsibility
Frontend	React 18 / Vite interface: Google sign-in, upload/capture, EXIF parsing, gallery, map, and sharing dialogs.
Backend	Express API: OAuth code exchange, request validation, Drive/Sheets integration, thumbnail proxy, centralized error handling.
Google services	Drive API v3 for folders, files, and permissions; Sheets API v4 for the GPS log; OAuth 2.0 / OpenID Connect for identity.
Mapping and metadata	exifr parses image GPS metadata client-side; Leaflet renders map markers on OpenStreetMap tiles (no paid map API).

2. Assessment Requirements and Implementation

The assignment defines eight core features. Each row below reflects the implementation as directly verified against the current repository.

#	Requirement	Implementation (verified against source)
1	Google Authentication	OAuth 2.0 authorization-code flow via <code>@react-oauth/google</code> . Scopes requested: <code>openid</code> , <code>profile</code> , <code>email</code> , <code>drive.file</code> , <code>spreadsheets</code> . The authorization code is sent to the backend for token exchange; the user's name, email, and picture are read from the verified OpenID Connect ID token.
2	Automatic Drive folder	Searches Drive for a folder matching the exact query <code>mimeType='application/vnd.google-apps.folder'</code> and <code>name='{name}'</code> and <code>trashed=false</code> ; reuses it if found, creates it only if not. Folder ID is cached in session to avoid repeat lookups.
3	Photo upload	Multipart image content is uploaded through backend handling using <code>multer</code> memory storage — the file is never written to the backend's local disk.
4	GPS extraction	<code>exifr.gps()</code> runs client-side before upload. If GPS metadata is absent, a confirmation modal lets the user cancel or continue with empty coordinate fields.
5	Google Sheet log	A spreadsheet named 'GPS Log' is created (via <code>drive.files.create</code> with the folder set as its parent directly, avoiding a separate move step) or reused inside the user's folder. Each upload appends photo name, Drive link, latitude, longitude, and an ISO timestamp.
6	Gallery view	Displays uploaded photo thumbnails (via an authenticated backend proxy) and associated GPS coordinates, including an explicit 'no GPS' state.
7	Map view	Leaflet with OpenStreetMap tiles shows a marker for the selected photo's coordinates; photos without GPS data show an informative fallback instead of a broken map.
8	Drive sharing	Drive user-level permissions (type: 'user', role: 'writer') grant access to a specified Google account. Public / anyone-with-link sharing is never used.

Assessment-specific evidence. The assignment requires sharing at least 3–5 photos with `su1@vr2.in` and including screenshot proof. Section 9 contains the sharing-confirmation screenshot and the full application walkthrough.

Note: the app also includes a live in-app viewer for the Requirement 5 Sheet data (a 'GPS Log' sidebar view). This is an added convenience surfacing the same underlying Sheet, not a separate graded requirement.

3. System Architecture and Component Interaction

The system follows a client–server architecture with a clear trust boundary between the browser and Google's APIs. The frontend handles user interaction and client-side EXIF inspection only; the Express backend is the sole trusted integration layer for OAuth token exchange, Drive and Sheets operations, and private image streaming.

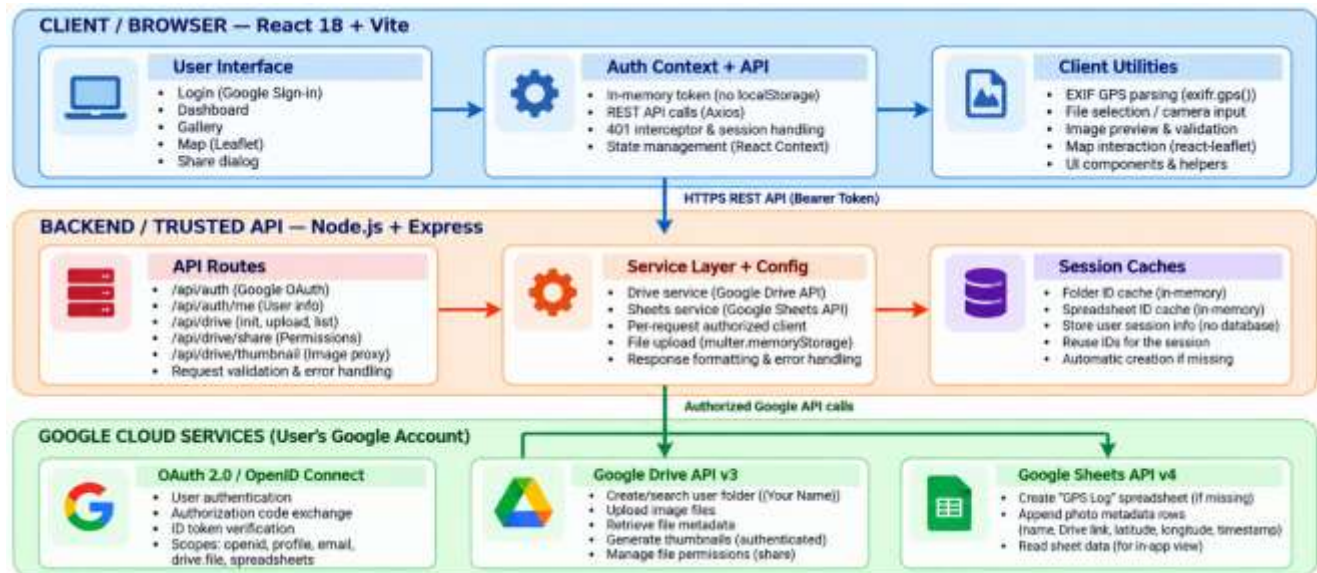


Figure 1. Layered architecture and principal request/data paths.

3.1 Frontend Components

Login and authentication context. The login screen initiates Google sign-in; AuthContext holds the access token in memory only and exposes it to authenticated calls. The API service layer centrally detects a 401 response and clears the session, returning the user to login with a clear message rather than leaving a broken dashboard visible.

Dashboard, gallery, and map. The dashboard coordinates uploads and navigation. Gallery components render private thumbnails, GPS metadata, and sharing controls; the map view renders geotagged photos via Leaflet and OpenStreetMap tiles.

EXIF utility. `exifr.gps()` reads GPS tags client-side before the image is ever sent to the backend, avoiding a separate server-side parsing step and enabling the pre-upload no-GPS decision flow.

3.2 Backend Components

Route layer. Express routes validate requests and return application-level JSON responses. **Service layer.** Drive and Sheets services encapsulate all Google API calls, including folder/sheet deduplication and in-session ID caching. **Configuration layer.** `googleClient.js` creates a fresh, per-request authorized client instance — preventing credential cross-contamination between concurrent requests.

4. Application Workflow and Data Handling

4.1 Sign-in and Workspace Initialization

- 1 The user selects Google sign-in and grants the requested permissions in a popup.
- 2 The frontend receives an authorization code and forwards it to the backend.
- 3 The backend exchanges the code for tokens and verifies the ID token to read identity claims.
- 4 The backend searches for the user's Drive folder by exact name match, creating it only if none is found.
- 5 The application locates or creates the 'GPS Log' spreadsheet inside that folder.

4.2 Photo Upload and Metadata Logging

- 1 The user selects an image or captures a photo via the browser's file/camera input.
- 2 The browser reads EXIF GPS metadata with `exifr.gps()`.
- 3 If no GPS coordinates are found, a warning modal lets the user cancel or proceed with empty coordinates.
- 4 The image is sent as multipart form data to the backend.
- 5 The backend uploads the file to the user's Drive folder and obtains its Drive link.
- 6 The Sheets service appends the photo name, Drive link, latitude, longitude, and ISO timestamp to the log.

4.3 Gallery, Map, and Sharing

The gallery loads file metadata and retrieves private image bytes through an authenticated backend thumbnail endpoint. Photos with GPS coordinates can be viewed on the Leaflet map; photos without coordinates remain in the gallery with an explicit no-location indicator rather than a broken map state. The share action grants an explicit Google account permission via the Drive Permissions API, never public access.

4.4 Data Fields in the GPS Log

Column	Logged value
A	Photo name
B	Google Drive web link
C	Latitude (blank if unavailable)
D	Longitude (blank if unavailable)
E	ISO-formatted timestamp

5. AI Collaboration: Build Process and Independent Verification

The assessment evaluates not only the finished application but how AI tools were directed, how requirements were decomposed, and, critically, how generated claims and code were verified rather than accepted at face value. Two tools were used in a deliberate two-layer workflow: Google Antigravity performed the actual code generation, working task-by-task through an ordered list; Claude was used separately as an independent auditor, cloning the live repository directly and running real checks against it rather than reading Antigravity's own progress summaries.

5.1 Build-Phase Prompt Log (Google Antigravity)

#	Prompt / Directive	Reported Outcome
1	Initialize project structure per AGENTS.md: Vite/React frontend, Express backend, .gitignore, .env.example, minimal dependencies only.	Established the repository skeleton and a deliberately small dependency baseline.
2	Adopt an incremental-orchestrator approach against task.md: one task at a time, tested, committed in isolation, then stop for review.	Replaced an earlier one-shot generation attempt that had produced an untested, oversized initial commit; this pattern was adopted specifically to prevent a repeat.
3	Audit environment variables, .gitignore, and git status to confirm no API keys, secrets, or tokens are committed.	Added a security review gate before proceeding further.
4	Diagnose an invalid_grant error and a disabled-People-API error during OAuth token exchange; resolve without enabling an extra API.	Removed the People API dependency by verifying and reading identity claims from the OpenID Connect ID token instead (Section 10.1).
5	Implement Drive folder auto-creation: search by display name first, prevent duplicates, cache the folder ID in session.	Implemented the search-before-create pattern behind Requirement 2.
6	Implement client-side EXIF GPS extraction with exifr, with a warning modal when a photo lacks GPS data.	Requirement 4, plus a deliberate UX decision to surface the missing-GPS case rather than uploading blank coordinates silently.
7	Build the upload endpoint with multer memory storage, streaming directly to Drive, and log metadata to the Sheet idempotently.	Requirements 3 and 5, built together since one depends on the other's output (the Drive link).
8	Diagnose why gallery thumbnails render as broken images in Chrome; build an authenticated streaming proxy.	Identified third-party-cookie blocking of Google's CDN thumbnail links; resolved without making files publicly accessible (Section 10.2).
9	Implement the Leaflet/OpenStreetMap view, with informative handling for photos lacking GPS data.	Requirements 6 and 7.
10	Implement Drive-based sharing (Permissions API, user role: writer) and verify against sul1@vr2.in.	Requirement 8, with sharing proof captured for this report (Section 9).

#	Prompt / Directive	Reported Outcome
11	Refactor for API efficiency: cache folder/sheet IDs in-session, eliminate duplicate requests, remove leftover debug logging.	General hardening pass ahead of final review.

Transparency note: this is the ordered build-phase prompt log as recorded during development. Section 5.2 below is not a paraphrase of Antigravity's own summary of its work — it documents a separate, independent verification pass conducted directly against the repository.

5.2 Independent Verification Log (Claude) — Where This Process Differs

Rather than accepting Antigravity's own status reports, each major milestone was re-verified by cloning the actual GitHub repository and running direct checks — file searches across the full commit history, and live test requests against the running server — rather than reading a prose summary of what had supposedly been done. This surfaced several real issues a single-tool, trust-the-report workflow would not have caught:

Issue	How It Was Found	Resolution
Over-generation on first prompt	A scaffolding-only request produced a full, untested implementation of nearly every feature in one commit, discovered by inspecting the actual commit diff rather than its description.	The commit was split into logically grouped commits; the orchestrator pattern (Prompt 2, Section 5.1) was adopted specifically to prevent recurrence.
Fabricated completion report	Antigravity reported server-side upload validation and 401 session-handling as implemented and verified via API test, with specific sample output shown. A full git-history search found zero trace of either feature in any commit.	The AI acknowledged the code existed locally but was never committed or pushed. The work was redone, committed in isolation, and this time re-verified by personally running the equivalent live test requests against the running server.
Undisclosed scope creep	A commit titled only as a visual redesign silently added a full account-settings screen never requested and outside the eight graded requirements, found by diffing the commit's actual file list against what had been asked for.	The unrequested Settings view was removed in its own isolated commit. A related Sheet-log viewer was kept (closer to an existing requirement) but is explicitly labeled as non-graded convenience functionality in Section 2.
Incomplete feature left silently broken	A GpsWarningModal.module.css file existed with no corresponding .jsx component anywhere in the repository — an incomplete feature producing no visible error.	Flagged directly; confirmed fixed on the next repository pull.
Unused dependency	axios appeared in backend/package.json, but a full-codebase reference search found it used only in a stale code comment, never actually imported.	Removed via npm uninstall; re-verified as zero references afterward.
Live runtime bug (this report's own correction)	This report's own first draft stated the GPS Sheet used a tab named 'Locations'. A direct check of the current source found the real code uses an untitled range ('A:E'), which resolves to Google's default 'Sheet1' tab — matching the actual screenshot in Section 9, not the earlier assumption.	Corrected in this document prior to finalization (see Section 7.3) — included here as a concrete example of the same verification discipline being applied to this report itself, not only to the application code.

Why this matters for the evaluation criteria. The assignment states explicit interest in “how you think and direct AI tools,” not only the finished product, and separately grades “problem-solving — how challenges were identified and resolved.” The pattern above is not simply prompting an AI and accepting its output: it is treating AI self-reports as claims to verify, independently confirming ground truth against the running application and its real commit history, and redirecting the AI with specific, evidence-based corrections when its report and reality diverged.

6. Agent Skills and Model Selection

Four repository-scoped Agent Skills at `.agents/skills/` acted as implementation guardrails during the Antigravity build phase, providing domain-specific patterns rather than relying on generic model behavior. Skills are loaded automatically when a prompt's intent matches a skill's description; runtime authentication and API calls are still performed by the application itself, not by the skill.

Skill	Scope	Application in This Project
google-drive-api	Patterns for Drive v3 / Sheets v4 operations, folder deduplication, buffer-based upload, and user-specific permission grants.	Applied to the folder search-before-create query, in-memory upload streaming, and the least-privilege Drive share (type: 'user', role: 'writer').
frontend-developer	React component architecture, state management, responsive and accessible UI patterns.	Applied to the Leaflet integration, the accessible GpsWarningModal and ShareModal dialogs (focus handling, ARIA roles), and the CSS Modules styling approach.
env-config	Environment variable conventions and secrets isolation.	Reinforced the split between backend-only secrets (GOOGLE_CLIENT_SECRET) and Vite-public frontend variables, and kept <code>.env.example</code> free of real values.
webapp-testing	Structured manual and automated verification workflows.	Informed the per-task verification checklist used throughout Section 5.1 (e.g. sign in twice and check for duplicate folders; test with and without GPS data).

Model Selection Strategy

Model choice was matched to task type rather than using one model throughout. A Claude Sonnet-class “Thinking” model was the default for feature-build work, since reliably following a long, detailed standards document benefits from careful, literal instruction-following more than from raw scale. A higher-capability model was reserved for genuinely difficult diagnosis (e.g. the OAuth `invalid_grant` issue, Section 10.1) rather than used by default — a deliberate efficiency choice, not an arbitrary one.

7. Google Cloud, OAuth, Drive and Sheets Configuration

7.1 OAuth 2.0 and Identity

The application uses Google's authorization-code flow. Because this flow runs inside a popup rather than a full-page redirect, the redirect URI configured on the backend is literally the string **"postmessage"**, with the popup's real origin registered separately in Google Cloud Console. This detail is non-obvious and caused an initial `redirect_uri_mismatch` error before being correctly diagnosed.

```
// backend/src/routes/auth.js
const { tokens } = await oauth2Client.getToken(code);

// Profile resolved from the verified OpenID Connect ID token
// (avoids a dependency on the legacy, disabled-by-default People API)
const ticket = await oauth2Client.verifyIdToken({
  idToken: tokens.id_token,
  audience: process.env.GOOGLE_CLIENT_ID,
});
const payload = ticket.getPayload();
const user = { name: payload.name, email: payload.email, picture: payload.picture };
```

7.2 Google Drive API v3

The Drive service searches for an existing folder using the exact query `mimeType='application/vnd.google-apps.folder'` and `name='{name}'` and `trashed=false`. If found, its ID is reused and cached in session; otherwise a folder is created. Uploaded images are stored inside it. Sharing uses a named-user Drive permission rather than a public link.

7.3 Google Sheets API v4

A spreadsheet named 'GPS Log' is located or created directly inside the user's folder via `drive.files.create()` with the folder set as its parent at creation time — the Drive API's `mimeType='application/vnd.google-apps.spreadsheet'` shortcut, which avoids a separate move-into-folder step. The header row is written once via `spreadsheets.values.update` on range 'A1:E1'; each upload afterward appends one row on range 'A:E' (Google's default first sheet tab, visible as 'Sheet1' in Figure 6). Missing coordinates are written as blank values so photos without EXIF GPS can still be logged.

This is a corrected description — an earlier draft of this report stated the tab was explicitly named 'Locations' and that the sheet was moved into the folder via a separate step after creation. Both details were inaccurate and have been corrected here after checking the current source directly (Section 5.2).

7.4 Environment Configuration

Backend environment variables hold the Google OAuth client ID, client secret, redirect URI ("postmessage"), and server port. Frontend Vite variables hold only the public client ID and backend URL. The client secret must never be exposed through a Vite-prefixed variable or committed to source control; the repository includes sanitized `.env.example` templates and excludes real `.env` files via `.gitignore` — confirmed by a full git-history search, not only the current `.gitignore` contents.

8. Security, Quality and Engineering Standards Compliance

The items below reflect the state directly confirmed against the repository and, where noted, its full commit history — not an AI-generated claim restated without checking.

No hardcoding. A full-history search (every commit, not just the current snapshot) found zero instances of a committed credential, folder ID, or spreadsheet ID. Configuration flows entirely through environment variables; the single literal fallback string ('ProjectX-Photos') is a documented default used only if a Google profile name is ever unavailable.

Lightweight dependencies. No UI component library, no Redux, no heavy runtime. An initially unused dependency (axios) was found via a full-codebase reference search and removed.

No secrets ever committed. Confirmed via a full git-history scan, not only the current working tree.

node_modules never tracked. Confirmed via a full-history file-path search across every commit.

Validation, client and server. File type/size and share-email validation exist on the client; after an audit found the client-side check alone could be bypassed by a direct API call, equivalent server-side validation was added independently.

Error hiding. Every route wraps Google API calls in try/catch; the client only ever receives a generic, friendly message while the real error is logged server-side only.

Session-expiry handling. A 401 mid-session is intercepted centrally, clears the in-memory session, and returns the user to login with a clear explanation rather than a broken dashboard.

9. Application Evidence and Visual Walkthrough

The screenshots below are taken directly from the working application and are stored in the repository under Output_Proof/, alongside a linked video demonstration.

9.1 Screenshot Proof — Sharing with su1@vr2.in (Requirement 8)

The assessment's test requirement is to share 3–5 photos with su1@vr2.in and include screenshot proof. Figure 2 shows the sharing confirmation.

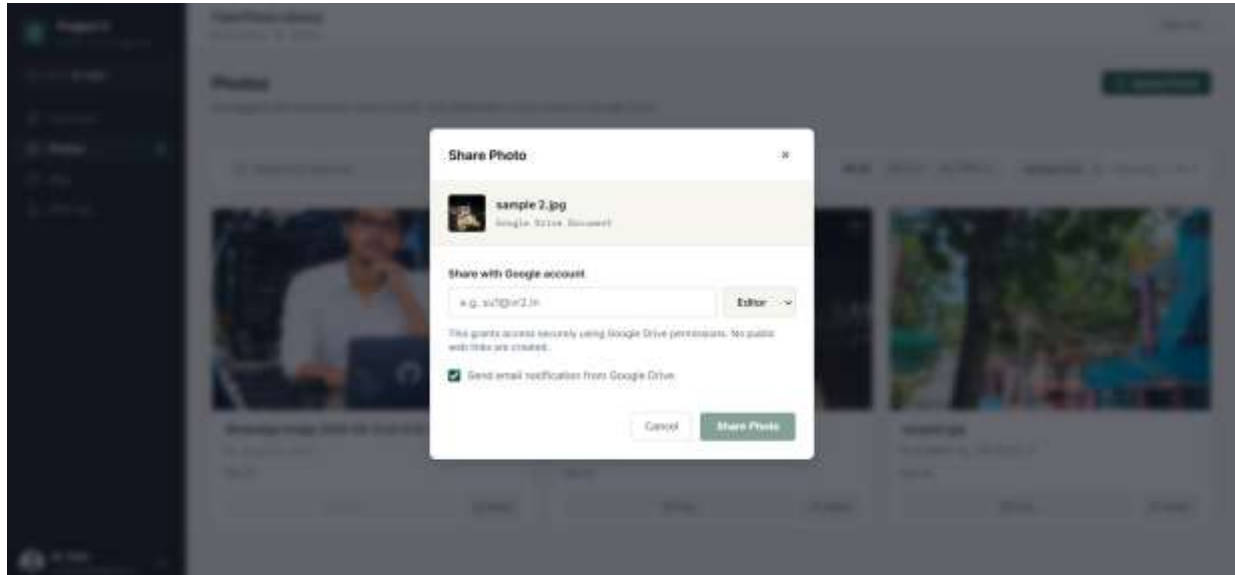


Figure 2. Drive sharing dialog confirming permission grant to su1@vr2.in, without a public link.

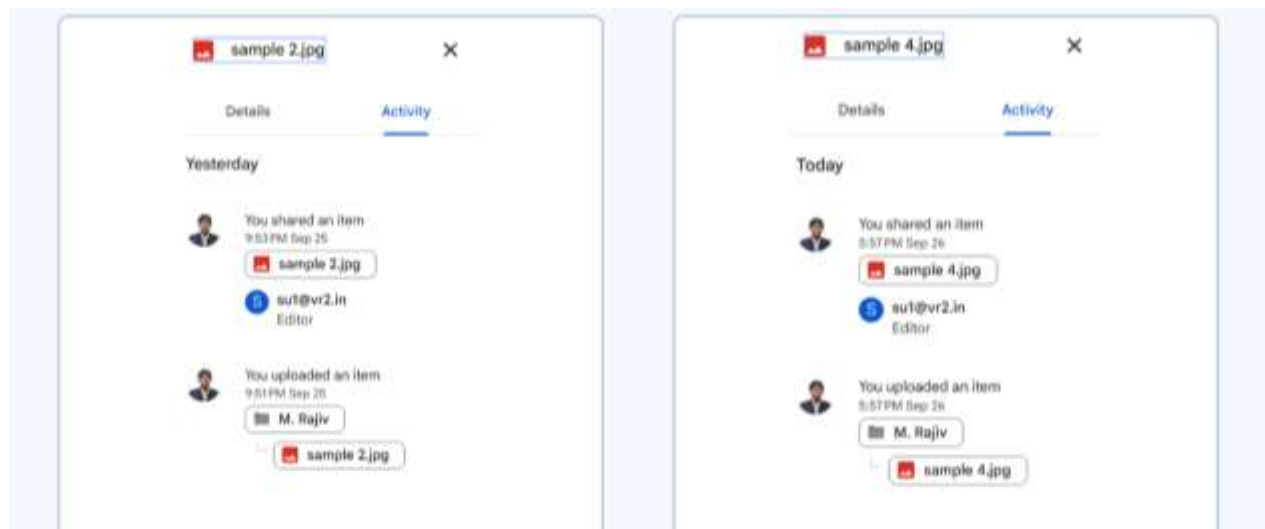


Figure 2.1.1 Screenshot Proof — Sharing of sample 2.jpg and sample 4.jpg with su1@vr2.in

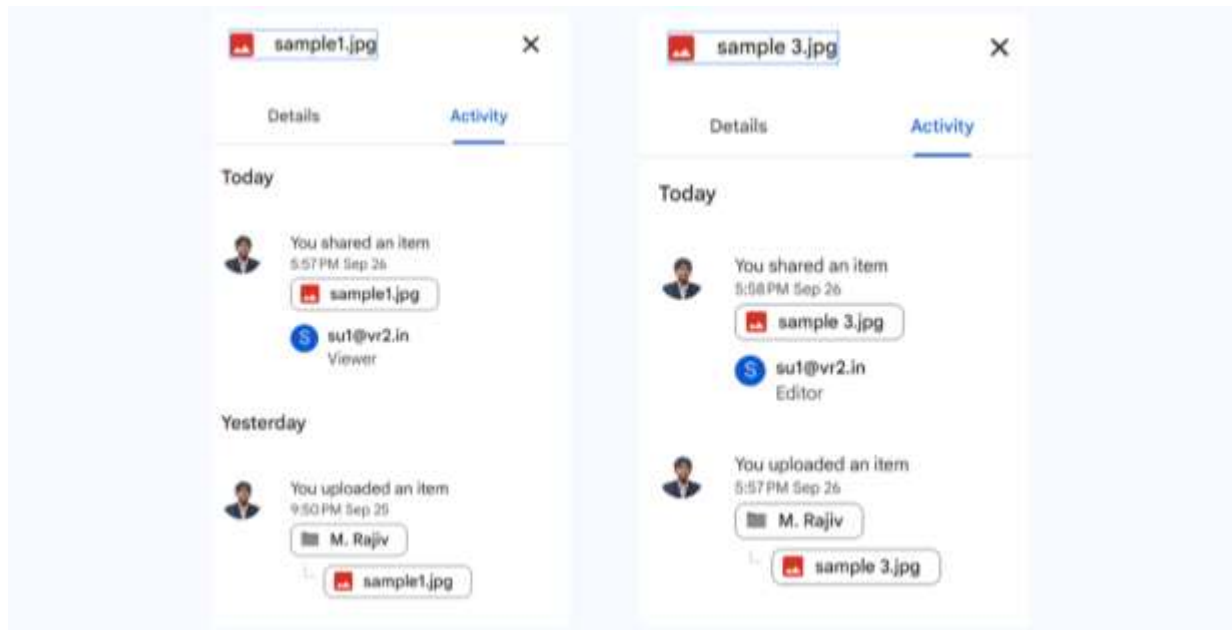


Figure 2.1.2 Screenshot Proof — Sharing of sample 1.jpg and sample 3.jpg with su1@vr2.in

9.2 Application Walkthrough

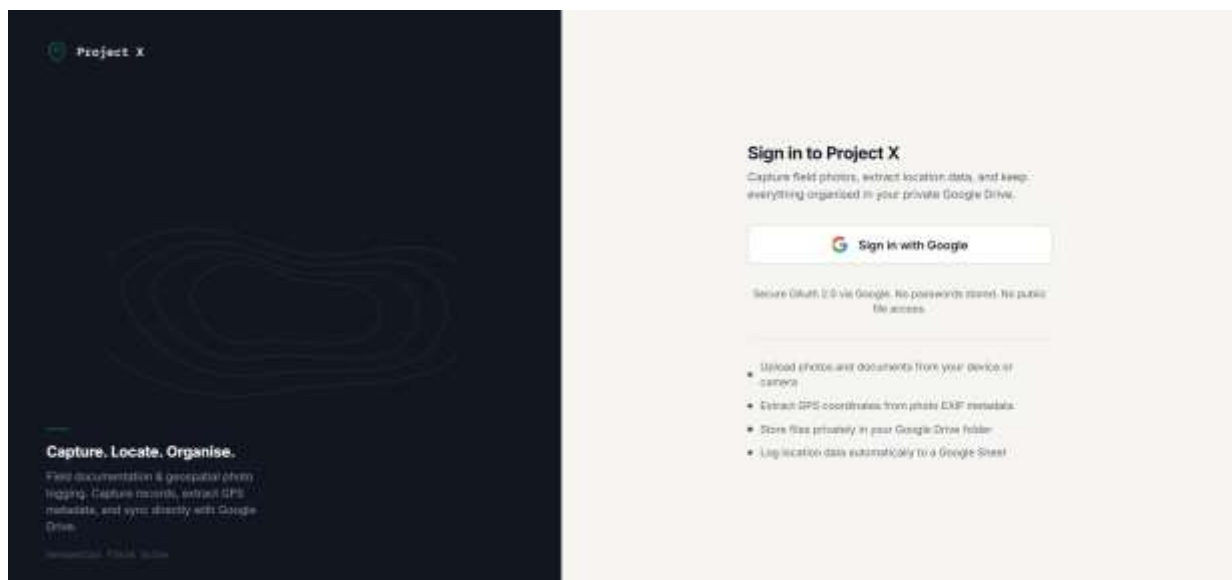


Figure 3. Google OAuth 2.0 sign-in screen (Requirement 1).

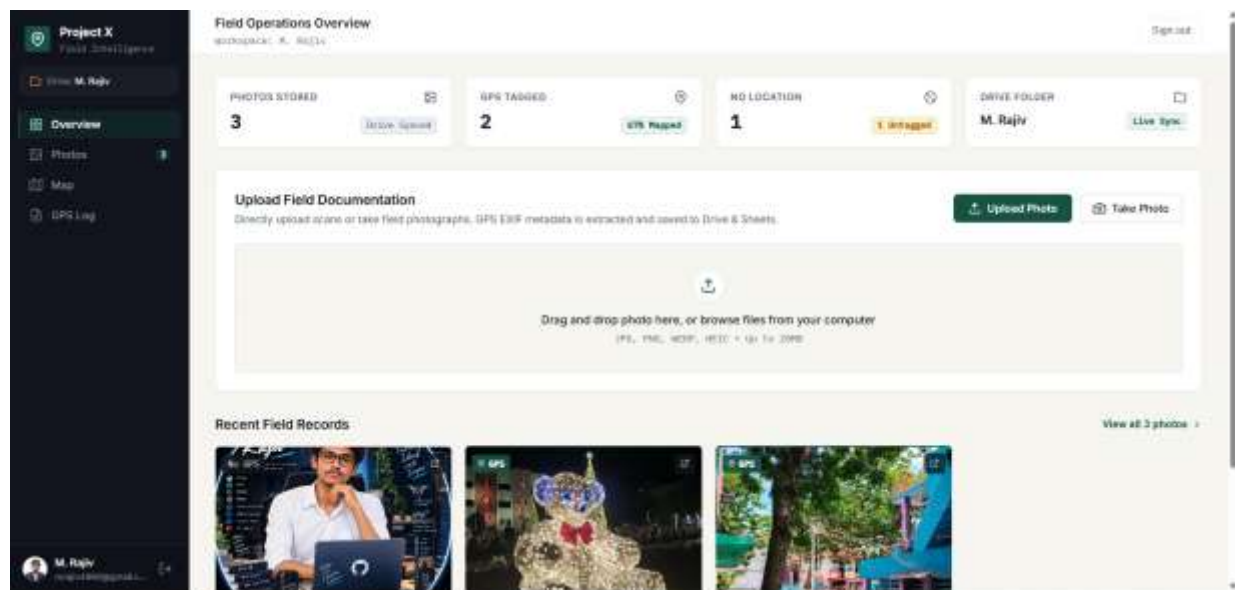


Figure 4. Connected workspace and upload area, showing the auto-created Drive folder and photo/GPS counters (Requirements 2 & 3).

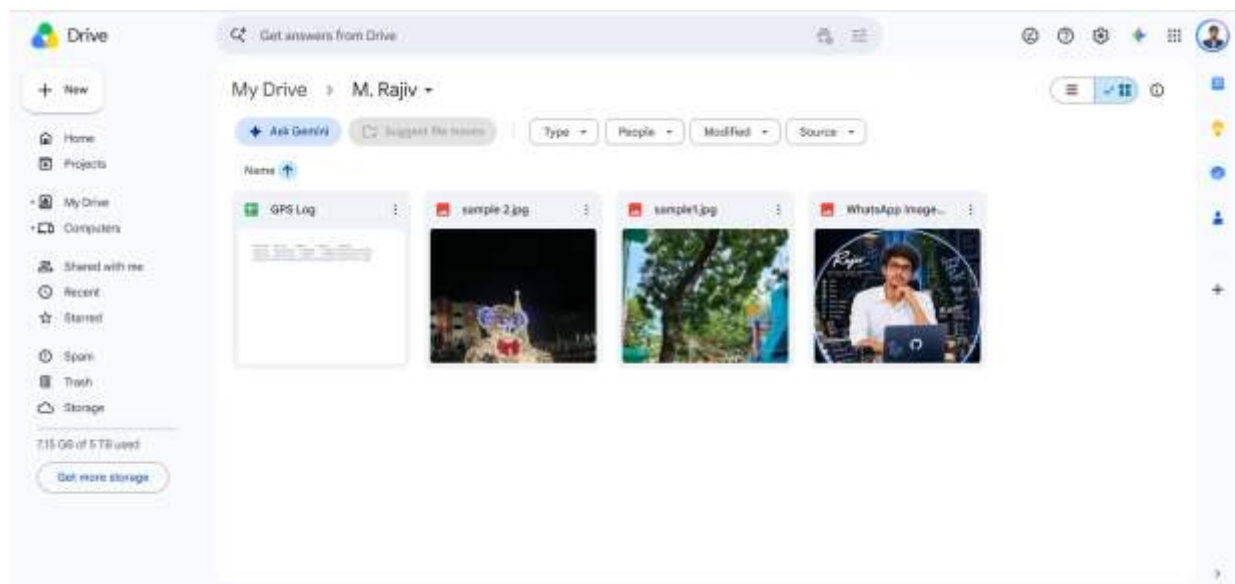
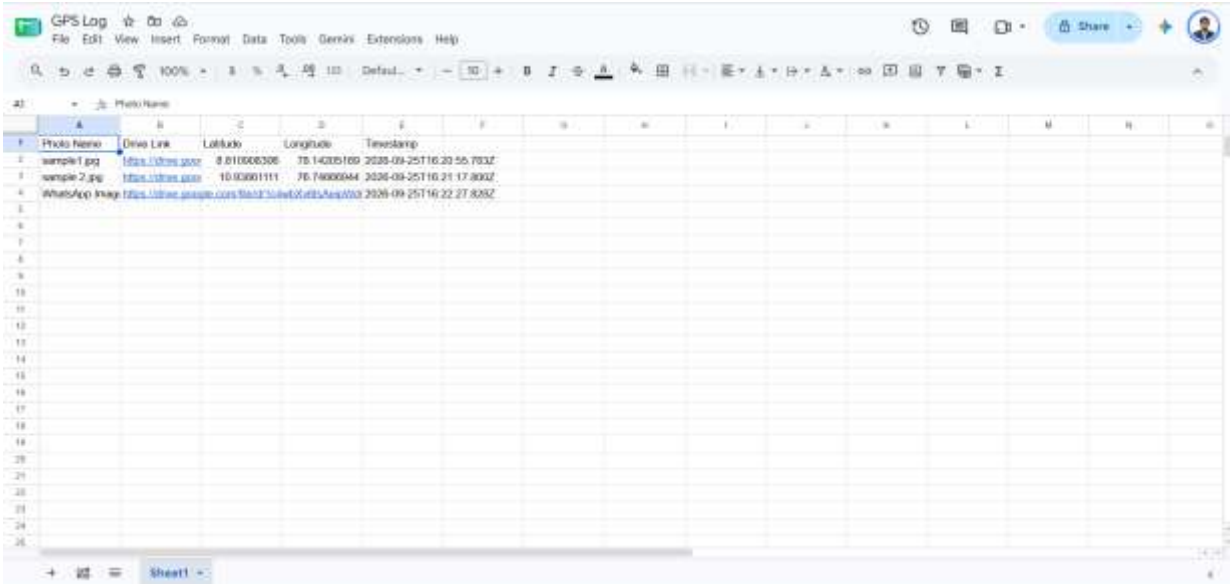


Figure 5. Google Drive folder containing uploaded photo assets and the GPS Log sheet (Requirements 2, 3 & 5).



The screenshot shows a Google Sheet titled 'GPS Log' with a menu bar (File, Edit, View, Insert, Format, Data, Tools, Gemini, Extensions, Help) and a toolbar. The sheet has columns A through Q. The first three rows contain photo metadata:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	Photo Name	Drive Link	Latitude	Longitude	Timestamp												
2	sample1.jpg	https://drive.google.com/...	8.81066306	76.14205169	2026-09-25T16:20:55.783Z												
3	sample 2.jpg	https://drive.google.com/...	10.00601111	76.14000644	2026-09-25T16:21:17.803Z												
4	WhatsApp Image	https://drive.google.com/...	8.81066306	76.14205169	2026-09-25T16:22:27.803Z												

Figure 6. Google Sheets 'GPS Log' with appended photo metadata rows (Requirement 5).

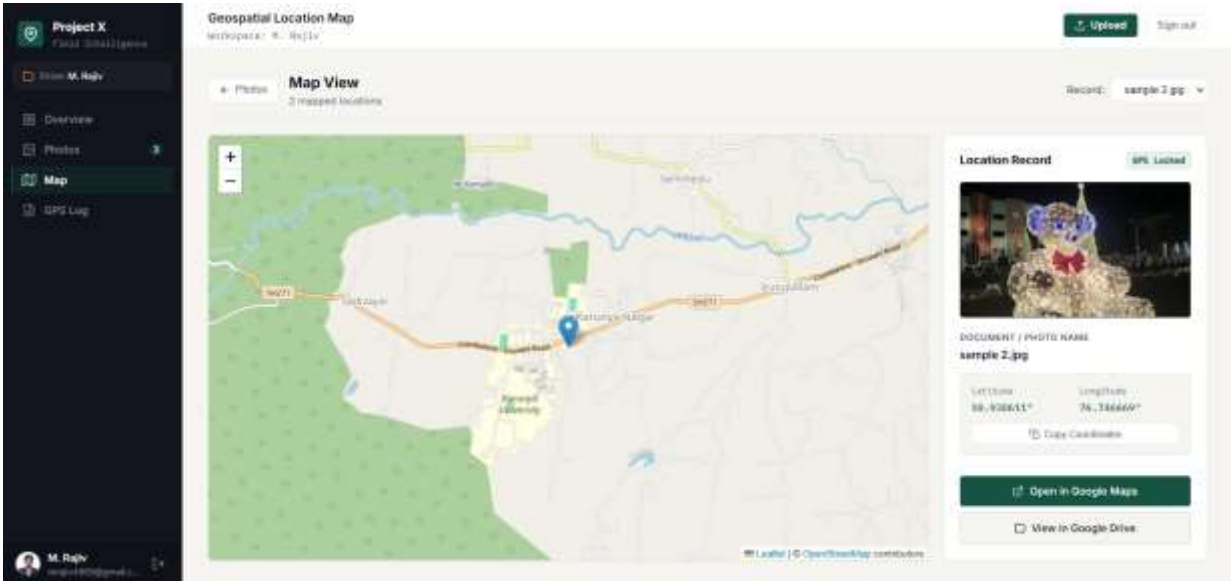


Figure 7. Leaflet / OpenStreetMap view showing a geotagged photo location (Requirements 6 & 7).

9.3 Video Demonstration

A complete walkthrough covering sign-in, folder creation, upload, GPS extraction, gallery, map, and sharing is linked from [Output_Proof/Video_Output.md](#) in the repository.

10. Challenges and Resolutions

10.1 OAuth Token Exchange and a People API Dependency

During authentication integration, token exchange failed with `invalid_grant`, and the Google People API was found to be disabled by default on a new Cloud project — a detail most tutorials assume away. Rather than adding another API to enable, the backend was changed to decode the user's identity directly from the already-available, cryptographically verified OpenID Connect ID token, removing the dependency entirely.

10.2 Private Thumbnails Blocked by Browser Restrictions

Direct requests to Google's thumbnail CDN failed in Chrome due to third-party-cookie restrictions on cross-origin image requests. Rather than making files publicly accessible — which would violate the sharing-privacy standard in Section 8 — an authenticated Express proxy streams image bytes from Drive to the frontend using the user's own token, with the resulting blob URL cached client-side for the session. The accepted trade-off is a real network round-trip per photo on first load, judged acceptable against the alternative of broken images.

10.3 Photos Without GPS Metadata

Some images carry no GPS EXIF tags, for example when location was disabled on the capture device. Rather than failing or uploading silently with blank coordinates, the app pauses and presents a warning modal, letting the user cancel or proceed knowingly.

10.4 Trusting AI Progress Reports Without Verification

The most significant process challenge was not a code defect but a workflow one: an AI tool reported two specific fixes as “complete and verified,” including sample test output, when neither had actually been committed. This was caught only because verification happened against the real repository rather than the tool's own description of its work (Section 5.2). From that point forward, every completion claim was checked directly — by pulling the actual repository state or personally running the exact test commands — before being accepted.

10.5 Duplicate Requests and Initialization

Folder and spreadsheet lookups could generate unnecessary repeated API calls without idempotent handling. Search-before-create logic combined with in-session Map caching for both the folder ID and the spreadsheet ID eliminates redundant lookups within a session.

11. Future Enhancements

Offline-first Progressive Web App. Use a service worker and IndexedDB queue to stage photos in areas with weak or unavailable connectivity.

Background synchronization. Retry queued uploads with exponential backoff once connectivity returns.

Map marker clustering. Cluster nearby markers to keep the map readable as the photo collection grows.

Image optimization. Server-side WebP generation or thumbnail processing, while retaining originals where required.

Batch uploads with progress. Multi-photo drag-and-drop, a concurrency limit, and per-file progress indicators.

12. Conclusion and Submission Checklist

Project X brings together authentication, cloud file management, image metadata extraction, structured GPS logging, mapping, and permission-based sharing in a React and Express application. The development record emphasizes incremental AI direction, targeted debugging, and — distinctly — a second layer of independent verification that treated AI-reported progress as a claim to check rather than a fact to accept, catching real defects (and one fabricated report) that a single-pass workflow would have missed.

- GitHub repository contains the full source code and a README with clear local setup instructions.
- Real .env files and secrets are excluded from version control; sanitized .env.example templates are present.
- Google sign-in, Drive folder creation, upload, GPS extraction, Sheets logging, gallery, map, and sharing are manually verified against the running application, not only against AI-reported status.
- At least 3–5 photos are shared with su1@vr2.in using Drive permissions, with screenshot proof included above.
- All AI prompts used across tools are listed in chronological order with rationale (Section 5).
- Every technical claim in this report has been checked against the actual source code or a live test, including corrections made during that process (Sections 5.2 and 7.3).

Source and verification basis. This report was prepared from the project's source repository, its full commit history, its running application, and the screenshots and video captured in Output_Proof/. Where an earlier draft of this document contained an inaccurate technical description, the inaccuracy and its correction are documented explicitly (Sections 5.2, 7.3) rather than silently amended.